



National Academy of Science and Technology (NAST)

(Affiliated to Pokhara University)

Accredited By University Grants Commission, (UGC), Nepal, (2022)

Uttar Behadi-04, Dhangadhi

Operating System

Research On Process & Threads



Submitted To

Mr. Kapil Dev Pant

Department of BCA



Submitted By

Student Name: Gajendra Awasthi

Program: Bachelor in Computer Application

Semester: Third Semester

Roll No: 16

ACKNOWLEDGMENT

I would like to express my sincere gratitude to everyone who supported me in completing this research on “Process & Threads”, prepared for the internal evaluation of the Bachelor of Computer Applications (BCA) program at Pokhara University.

I am especially thankful to my respected lecturer, Mr. Kapil Dev Pant, for his valuable guidance, constructive feedback, and continuous encouragement throughout the preparation of this research. His clear explanations and academic support greatly enhanced my understanding of operating system concepts.

I would also like to extend my appreciation to the BCA Department of National Academy of Science and Technology for providing the academic environment and resources necessary to complete this work successfully.

.....

Submitted By

Gajendra Awasthi
BCA Third Semester

.....

Submitted To

Mr. Kapil dev Pant
(Lecturer of Operating System)

Table of Contents

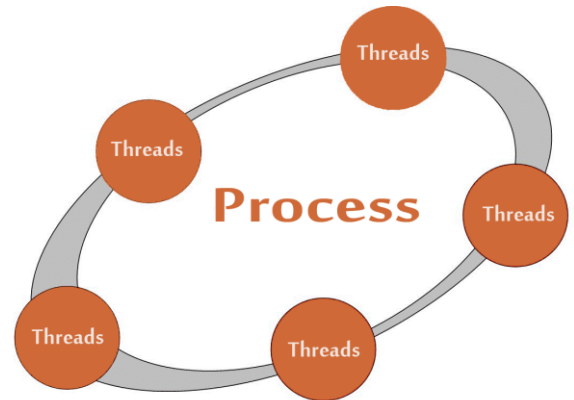
Chapter 1: Introduction To Process & Threads	1
1.1. Background.....	1
1.1.1 Why Study Processes and Threads?.....	1
1.1.2 Why Are They Important in Modern Computing?	2
1.2 Objectives of the Research.....	4
1.3 Scope & Limitation.....	5
1.4 Research Methodology	5
Chapter 2: Fundamentals of Process & its management	7
2.1 What is Process?	7
2.2 Characteristics of a Process	8
2.3 Process creation and termination	8
Chapter 3: Process Control Block (PCB)	9
3.1 Introduction to PCB.....	9
3.2 Structure of PCB.....	9
3.3 Role of PCB in Context Switching.....	11
Chapter 4: Inter-Process communication (IPC)	13
4.1 Introduction and need of IPC	13
4.2 Types of IPC	14
4.3 Difference Between Synchronous and Asynchronous Inter-Process Communication	15
4.4 Case Study: IPC in a Web Server.....	16
4.5 Advantages of IPC	17
4.6 Disadvantages of IPC.....	17
Chapter 5: Process Operations and types.....	18
5.1 Process Operations.....	18
5.2 Process States in Detail	18
5.3 Types of Processes.....	19
5.3.1 System vs User Processes.....	19
5.3.2 Foreground vs Background Processes	20
Chapter 6: Threads in Operating Systems	21
6.1 What is a Thread?.....	21
6.2 Characteristics of Threads	21
6.3 Thread States.....	23
6.4 Thread Operations.....	24
Chapter 7: Types of Threads and Threading Models.....	26
7.1 UserLevel Threads (ULTs).....	26
7.2 KernelLevel Threads (KLTs).....	27
7.3 Hybrid (ManytoMany) Model	27
Chapter 8: Advantages of Threads over Processes	28

Chapter 9: Challenges in Thread Programming.....	29
Chapter 10: RealWorld Examples and Case Studies.....	32
Chapter 11: Process vs Thread:.....	35
Chapter 12: Conclusion and Future Trends in Process and Threads.....	36

Chapter 1: Introduction To Process & Threads

1.1. Background

Processes and threads are very important ideas in operating systems. They help us understand how programs actually run on a computer. If we want to become good programmers, system designers, or IT professionals, we must understand these concepts clearly.



1.1.1 Why Study Processes and Threads?

1. They are the heart of the OS

The operating system (like Windows, Linux, or Android) uses processes and threads to run programs. Without understanding them, we cannot really understand how the OS works, how it runs many programs at once, or how it keeps one program from crashing another.

2. Needed for real programming

In real software (like servers, apps, games), we use system calls like `fork()` (to create a process) or `pthread_create()` (to create a thread). Knowing when to use a process and when to use a thread helps us write faster and safer code.

3. Helps make programs faster and smoother

- If we use only one process, the program can do only one thing at a time.
- If we use multiple processes or threads, the program can do many things at the same time (for example: download a file, play music, and show the UI all together).
- Choosing the right one (process vs thread) makes the program fast and responsive.

4. Helps avoid big bugs

Many hard bugs (like crashes, wrong data, or programs freezing) happen because of mistakes in using processes and threads. If we understand them well, we can:

- Avoid race conditions (when two threads change the same data at the same time).
- Avoid deadlocks (when two threads wait for each other forever).
- Write code that is safe and stable.

5. Opens the door to advanced topics

Once we understand processes and threads, it becomes easier to learn:

- How the CPU schedules tasks.
- How memory is managed.
- How networks and distributed systems work.
- How cloud and microservices are built.

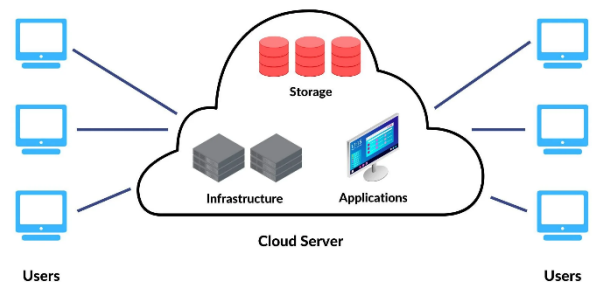
1.1.2 Why Are They Important in Modern Computing?

Processes and threads are used in almost every kind of computer system today.

1. Servers (Web, Database, Cloud)

Web Servers (like Apache, Nginx)

- When many users visit a website at the same time, the server uses many processes or threads to handle each user's request.
- This makes the website fast and able to serve many users at once.



Database Servers (like MySQL, PostgreSQL)

- Each user or app that connects to the database is handled by a separate thread or process.
- This allows many queries to run at the same time without slowing down.

Cloud and Microservices

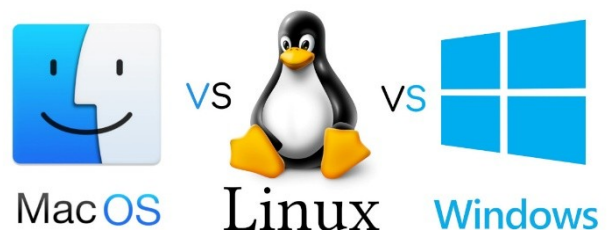
- In cloud systems, each service (like login, payment, or search) runs as a separate process (or container).
- Inside each service, threads handle many requests at the same time.

2. Desktop and Laptop Computers

Operating System (Windows, Linux, macOS)

The OS itself runs many small programs (called processes) for things like:

- Starting the computer.
- Managing files and devices.
- Running background tasks (like updates or antivirus).



These processes keep the system running smoothly.

Applications (Browsers, Office, Games)

Web Browsers (Chrome, Firefox)

- Each tab or website often runs in its own process so that if one tab crashes, others keep working.
- Inside each tab, threads handle loading pages, running JavaScript, and showing the UI.

Office Apps (Word, Excel, LibreOffice)

- One thread shows the window and buttons.
- Another thread saves the file in the background so the app doesn't freeze.



Games

- One thread draws the graphics (screen).
- Another thread handles the game logic (like movement and AI).
- Another thread handles sound and music.

3. Mobile Phones (Android, iOS)

Apps Run in Separate Processes

- Each app (WhatsApp, YouTube, Instagram) runs in its own process.
- If one app crashes, it usually does not crash the whole phone.



Background Work

- Even when an app is closed, it can run in the background (for example: playing music, downloading updates, or showing notifications).
- This is done using background processes or services.

Threads Make Apps Smooth

- When you open an app, one thread shows the screen.
- Another thread downloads data from the internet.
- This way, the app does not freeze while waiting for the internet.

4. Small Devices (Embedded and RealTime Systems)



Routers, Smart TVs, IoT Devices

These devices run many small processes or threads to:

- Handle Wi-Fi and internet.
- Show the screen or menu.
- Read sensors (like temperature or motion).

Because these devices have less power, threads are often used to save memory and CPU.

Cars, Robots, and Drones

In cars, one thread reads sensors (like speed and brakes).

Another thread controls the engine or brakes.

In drones, threads handle:

- Camera and video.
- GPS and navigation.
- Remote control from the user.

1.2 Objectives of the Research

- To understand the concept, characteristics, and lifecycle of a process.
- To study the Process Control Block (PCB) and its components.
- To analyze InterProcess Communication (IPC) mechanisms and their types.
- To explore process operations, states, and types

1.3 Scope & Limitation.

Scope

Focuses on processes and threads in normal operating systems (Linux, Windows, Unix-like).

This Research Explains:

- ✓ What is a process, its states, PCB, creation, termination.
- ✓ How processes talk to each other (IPC: shared memory, pipes, message passing, sockets).
- ✓ Types of processes (system/user, foreground/background, I/O-bound/CPU-bound).
- ✓ What is a thread, its types (user-level, kernel-level, hybrid), and how it is different from a process.
- ✓ Why threads are better than processes (faster, less memory, easy communication).
- ✓ Problems in thread programming (race condition, deadlock, debugging).
- ✓ Real examples from servers, desktop apps, mobile apps, and small devices.
- ✓ Uses simple code examples (C/Python) to show how processes and threads work.

Limitation

- ✓ Does not cover real-time OS (RTOS) in deep detail.
- ✓ Does not go into very low-level hardware details (CPU, MMU, etc.).
- ✓ Does not cover modern models like coroutines, async/await, or green threads in detail.
- ✓ Focuses on classical process/thread models, not very new or experimental OS designs.
- ✓ No heavy benchmarking or performance testing on many machines.
- ✓

1.4 Research Methodology

Literature Survey

- Study standard OS books (Silberschatz, Tanenbaum, etc.).
- Use trusted online sources (GeeksforGeeks, university notes, official docs).
- Read research papers on process/thread management and IPC.

Conceptual Analysis

- Break each topic into small, simple parts.
- Use diagrams (process states, PCB, thread vs process) to explain clearly.
- Use tables to compare (process vs thread, IPC methods, threading models).

Real-World Examples & Case Studies

Show how processes/threads are used in:

- Web browsers (Chrome, Firefox).
- Web servers and databases.
- Mobile apps (Android/iOS).
- Small devices (routers, IoT, embedded systems).

Give simple code examples:

- Creating a process
- Creating a thread.
- Simple IPC.

Chapter 2: Fundamentals of Process & its management

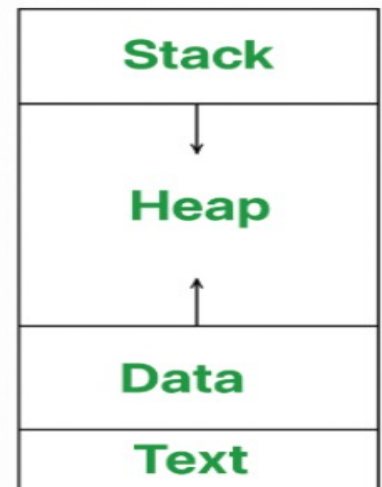
2.1 What is Process?

A process is a program in execution. For example, when we write a program in C or C++ and compile it, the compiler creates binary code. The original code and binary code are both programs. When we actually run the binary code, it becomes a process. A process is an 'active' entity instead of a program, which is considered a 'passive' entity.

A single program can create many processes when run multiple times; for example, when we open a .exe or binary file multiple times, multiple instances begin (multiple processes are created). A process in memory is divided into several distinct sections, each serving a different purpose

How the process in memory looks typically

- **Text Section:** A text or code segment contains executable instructions. It is typically a read only section
- **Stack:** The stack contains temporary data, such as function parameters, returns addresses, and local variables.
- **Data Section:** Contains the global variable.
- **Heap Section:** Dynamically memory allocated to process during its run time.



Difference Between process and program

Program	Process
A set of instructions stored on disk	A program in execution
Passive entity	Active entity
Does not need resources	Requires CPU, memory, I/O, etc.
Static (does not change)	Dynamic (changes state during execution)
Exists before execution	Exists only during execution
Example: .exe file	Example: Running application

2.2 Characteristics of a Process

- Unique Process ID (PID) – Identifies the process uniquely
- State – New, Ready, Running, Waiting, Terminated
- Program Counter – Address of the next instruction to execute
- CPU Registers – Store intermediate values during execution
- Memory – Includes code, data, heap, and stack
- Resources – Uses CPU time, files, and I/O devices
- Priority – Determines scheduling order

2.3 Process creation and termination

Process Creation

Process termination occurs when a process finishes execution or is stopped by the operating system.

How The Process Is Created,

- Assign a unique Process ID (PID) to the new process.
- Allocate memory for program code, data, stack, and heap.
- Initialize the Process Control Block (PCB), which contains:

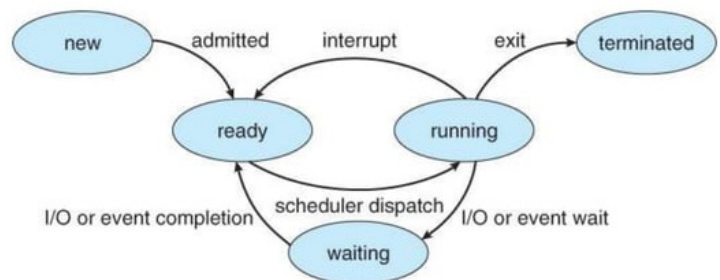
Process state

Program counter

CPU registers

Scheduling information

- Load the program into memory
- Set up resources, such as files, I/O devices, and security attributes.
- Insert the process into the ready queue for scheduling.



Process Termination

Process creation is the procedure by which a new process is generated in the system. A process may create other processes during its execution, forming a **parent–child relationship**.

How The Process Is Terminated,

- Process releases all **allocated resources** (memory, files, I/O devices).
- OS updates process status to **terminated**.
- PCB is removed from the process table.
- Parent process is notified of termination (exit status).
- Process is completely removed from the system.

Chapter 3: Process Control Block (PCB)

3.1 Introduction to PCB

A **Process Control Block (PCB)** is a data structure maintained by the **Operating System** to store all information related to a process. Whenever a process is created, the OS creates a PCB for that process, and when the process terminates, its PCB is removed.

The PCB acts as a **process identity card**, enabling the OS to manage, schedule, and control multiple processes efficiently in a multiprogramming environment. Without PCB, the OS would not be able to perform **context switching** between processes.

Each process in the system has **exactly one PCB**, which contains all the information required to pause and resume a process.

3.2 Structure of PCB

The structure of a PCB consists of several important fields that store detailed information about a process:

1. Process Identification Information

- Process ID (PID)
- Parent Process ID (PPID)
- User ID

2. Process State

- Current state of the process (New, Ready, Running, Waiting, Terminated)

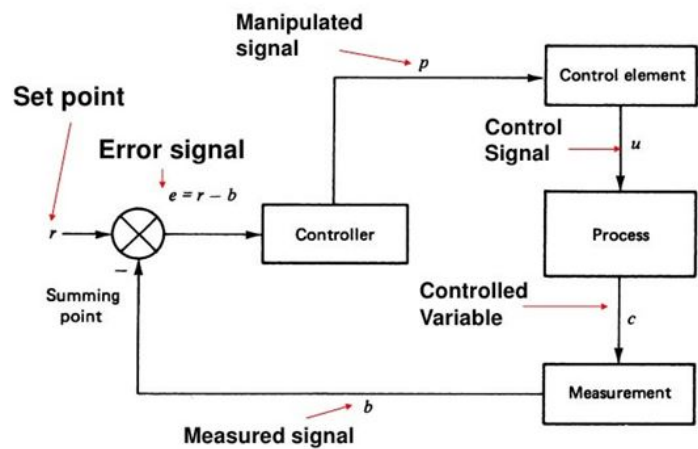
3. Program Counter

- Stores the address of the next instruction to be executed

4. CPU Registers

- General-purpose registers
- Stack pointer
- Accumulator
- Status registers

Process Control Block Diagram



5. CPU Scheduling Information

- Process priority
- Scheduling queue pointers
- Time slice information

6. Memory Management Information

- Base and limit registers
- Page table or segment table pointers
- Memory allocated to the process

7. Accounting Information

- CPU time used
- Process execution time
- Time limits

8. I/O Status Information

- List of open files
- I/O devices allocated to the process
- Pending I/O operations

3.3 Role of PCB in Context Switching

Context switching is the process of saving the state of a currently running process and restoring the state of another process so that multitasking can be achieved in an operating system.

The **Process Control Block (PCB)** plays a **central role** in context switching because it stores all the information required to stop and later resume a process.

How PCB Supports Context Switching

1. Saving Process State

- When a context switch occurs, the OS saves the current process's CPU state (program counter, registers, stack pointer, etc.) into its PCB.
- This ensures the process can resume execution from the exact point it was interrupted.

2. Storing Process Information

- PCB holds critical data such as process state, scheduling information, and memory details.
- This information helps the OS manage and track the process efficiently.

3. Loading Next Process State

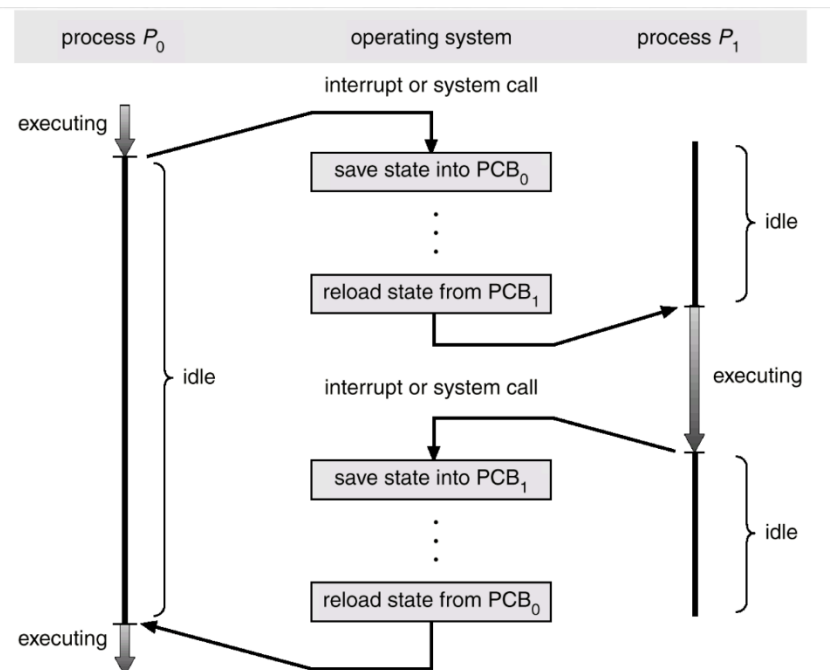
- The OS selects another process from the ready queue.
- The saved CPU context from the selected process's PCB is loaded back into the CPU.

4. Updating Process State

- The current process state is changed from **Running** to **Ready** or **Waiting**.
- The selected process state is updated from **Ready** to **Running** in its PCB.

5. Enabling Multitasking

- By storing and restoring process contexts through PCB, the OS allows multiple processes to share the CPU without interfering with each other.



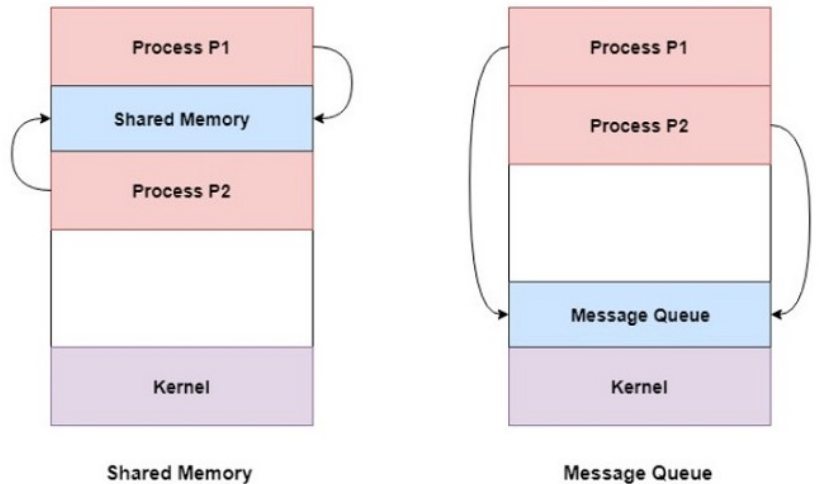
Why PCB Is Essential in Context Switching

- Prevents loss of process execution data
- Ensures accurate resumption of processes
- Supports fast and efficient CPU scheduling
- Maintains system stability and consistency

Chapter 4: Inter-Process communication (IPC)

4.1 Introduction and need of IPC

Inter-Process Communication (IPC) is a mechanism that allows **processes to communicate and synchronize** with each other while executing in an operating system. Since processes run in **separate memory spaces**, IPC provides a controlled way to exchange data and coordinate actions between them.



IPC is essential in **multiprogramming, multitasking, and distributed systems**, where multiple processes must work together to achieve a common goal.

Need of IPC

1. Information Sharing

- Processes often need to share data (e.g., producer–consumer problem).
- IPC enables safe and efficient data exchange.

2. Process Synchronization

- Ensures correct execution order among processes.
- Prevents race conditions and data inconsistency.

3. Resource Sharing

- Allows multiple processes to share system resources like files, memory, and devices.
- Ensures controlled and fair access to resources.

4. Modularity

- Large tasks can be divided into smaller cooperating processes.
- Improves system design and maintainability.

5. Performance Improvement

- Enables parallel execution of tasks.
- Increases CPU utilization and system throughput.

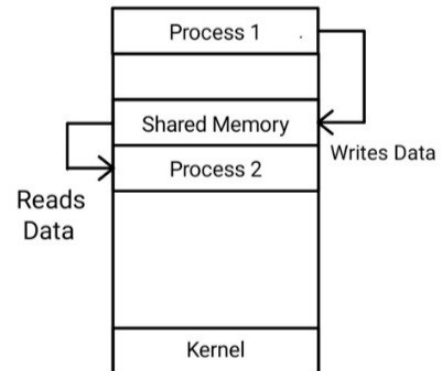
6. Event Notification

- One process can notify another about events (signals, messages).
- Useful in client–server systems.

4.2 Types of IPC

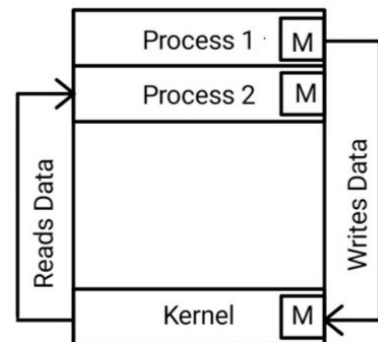
1. Shared Memory

- Multiple processes share a common memory region.
- Processes read and write data directly.
- Requires synchronization mechanisms like semaphores or mutexes.
- **Fastest IPC method.**



2. Message Passing

- Processes communicate by sending and receiving messages.
- Managed by the operating system.
- Safer but slower than shared memory.
- Used in client–server systems.



3. Pipes

- Provide one-way communication between processes.
- Can be unnamed (parent–child) or named (FIFO).
- Simple and commonly used in UNIX systems.

4. Sockets

- Enable communication between processes on the same or different systems.
- Use network protocols such as TCP or UDP.
- Commonly used in network-based applications.

5. Signals

- Used to notify a process about an event.
- Carry minimal information.
- Mainly used for control and interruption.

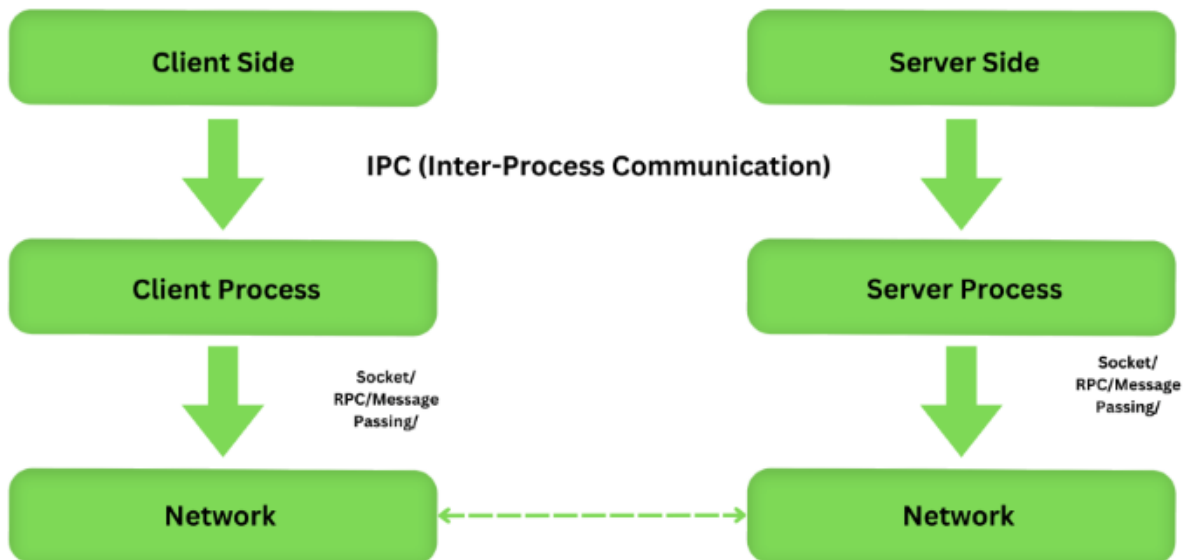
4.3 Difference Between Synchronous and Asynchronous Inter-Process Communication

S. No	Synchronous Inter-Process Communication	Asynchronous Inter-Process communication
1	It is also known as blocking inter-process communication in both send and receive modes.	It is also known as non-blocking inter-process communication in both send and receive modes.
2	Blocking send mode: The sending process is blocked until the message is received by the receiving process.	Non-block send mode: The message will be sent by the sending process and resumes the operation. It doesn't matter whether the receiver exists or not.
3	Blocking receive mode: In this, the receiver can block the available message and will not continue the process until the sending process exists.	Non-blocking receive mode: The receiver doesn't wait for the sender to send the message or data and continues the process. It receives either a NULL message or a valid transmitted message by the sender.
4	This communication occurs between various processes running at the same time.	In this communication, a process that receives asynchronous messages may or may not be used by the process it is trying to send a message to.

4.4 Case Study: IPC in a Web Server

Introduction

A **web server** is a system that handles client requests (usually from web browsers) and serves web pages or data over the internet. Modern web servers are designed to handle **multiple client requests simultaneously**, which requires the use of **multiple processes or threads**. To coordinate these processes efficiently, **Inter-Process Communication (IPC)** mechanisms are used.



Web Server Architecture

A typical web server consists of:

- **Master (Parent) Process** – manages configuration and spawns worker processes.
- **Worker Processes** – handle client requests.
- **Logging Process** – records access and error logs.
- **Cache Manager** – manages shared cached content.

These components must **communicate and synchronize**, making IPC essential.

Example: Request Handling Flow on WebServer

1. Client sends an HTTP request via **socket**.
2. Master process assigns the request to a worker process.
3. Worker checks **shared memory cache** for requested content.
4. Worker sends status updates using **message passing**.
5. Logs are sent through **pipes** to the logging process.
6. Signals manage process lifecycle and configuration reloads.

4.5 Advantages of IPC

- It helps to share the same data or information between the multiple processes and perform various tasks. In this, the process requires access to the remote process.
- The different processes can share the resources to provide communication between them.
- It increases the computational speed during communication between the processes.
- It enhances efficiency because the construction is divided into various modules. Where these modules cooperate using IPS.
- Privilege separation.
- Comfort.
- It helps the processes and OS to communicate with each other with synchronization.
- The communication is bidirectional.
- Allows control of one application with another application.
- Enables data sharing without any interference.

4.6 Disadvantages of IPC

- In message-passing IPC, a small amount of data is exchanged between the processes.
- The processes should not write to the same memory location while using a shared-memory method.
- Synchronization issues.
- Data protection issues.
- Data accessing issues.
- It doesn't work on multiple machines at the same time.

Chapter 5: Process Operations and types

5.1 Process Operations

Process operations refer to the actions performed by an operating system to manage the life cycle of a process. These operations ensure proper execution, coordination, and termination of processes in a multitasking environment.

5.2 Process States in Detail

A **process state** represents the current condition of a process during its execution lifecycle. As a process executes, it moves through different states, which are managed by the operating system to ensure efficient CPU utilization and multitasking.

1. New State

- The process is in the **new state** when it is being created.
- The operating system initializes the process and creates its **Process Control Block (PCB)**.
- Memory and required resources are allocated.

Example: A program just launched by the user.

2. Ready State

- The process is loaded into main memory and is ready to execute.
- It is waiting for CPU allocation by the scheduler.
- Multiple processes can be in the ready state simultaneously.

3. Running State

- The process is currently being executed by the CPU.
- At any given time, only one process can be in the running state on a single-core CPU.

4. Waiting / Blocked State

- The process cannot continue execution until a specific event occurs.
- Common reasons:
 - I/O operation
 - Waiting for a resource
 - Synchronization signal

5. Terminated (Exit) State

- The process has completed execution or has been terminated due to an error.
- The OS releases all allocated resources.
- The PCB is removed from the process table.

Additional Process States

6. Suspended Ready State

- The process is ready but temporarily removed from main memory.
- Stored in secondary storage due to memory shortage.

7. Suspended Blocked State

- The process is waiting for an event and is also swapped out of memory.

5.3 Types of Processes

5.3.1 System vs User Processes

Feature	System Process	User Process
Definition	A process created and managed by the operating system to perform system-level tasks.	A process created by a user to perform specific user-level tasks or applications.
Purpose	To manage hardware, system resources, and provide services to user processes.	To execute user programs like text editors, browsers, or games.
Privileges	Runs in kernel mode with high privileges.	Runs in user mode with limited privileges.
Creation	Automatically by the OS during system boot or service execution.	Created explicitly by the user.
Examples	File system manager, device drivers, memory manager.	Word processor, web browser, calculator.
Priority	Usually high priority for system stability and performance.	Usually lower priority than system processes.
Impact on System	Directly affects system stability and performance.	Primarily affects the user experience.
Communication	Can interact with hardware directly and with other processes via IPC.	Communicates with the OS for resource requests.

5.3.2 Foreground vs Background Processes

Feature	Foreground Process	Background Process
Definition	A process that interacts directly with the user through a terminal or GUI.	A process that runs without direct user interaction.
User Interaction	Requires user input or provides immediate output to the user.	Runs silently in the background; no direct user input required.
Priority	Usually higher priority because user expects immediate response.	Usually lower priority , as it performs tasks in the background.
Execution Control	Runs in the foreground ; user can pause, stop, or terminate it.	Runs independently in the background ; user usually unaware of its execution.
Examples	Text editor, web browser, command-line shell waiting for input.	File backup, virus scanning, printing services, system monitoring tasks.
Impact on System	Directly affects user experience; blocking may freeze interaction.	Affects system resources but not directly visible to the user.
Visibility	Process is visible to the user.	Process runs hidden from the user.

Chapter 6: Threads in Operating Systems

6.1 What is a Thread?

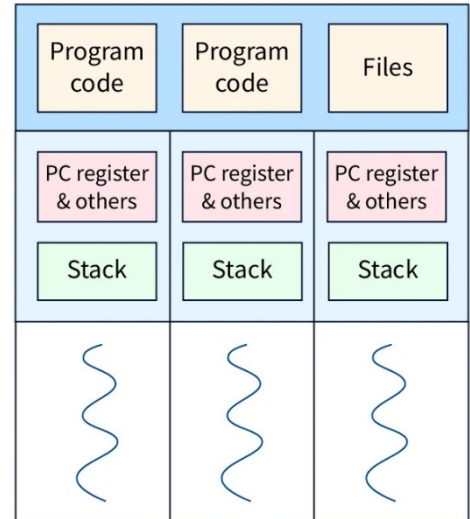
A **thread** is the **smallest unit of CPU execution** within a process. It represents a single sequence of instructions that the CPU can execute independently, but it shares the **process's resources** like memory, files, and data with other threads of the same process.

- A thread cannot exist independently; it always belongs to a process.
- Multiple threads in a process share the same memory and resources, which allows fast communication between them.
- Each thread has its own:

Program counter (PC)

Stack

Registers



6.2 Characteristics of Threads

➤ Smallest Unit of Execution

- A thread is the smallest sequence of programmed instructions that can be managed independently by the CPU.
- It is part of a process and cannot run independently outside it.

➤ Belongs to a Process

- Threads always exist **within a process**.
- Multiple threads can exist in the same process, sharing its resources.

➤ Shared Resources

- Threads in the same process share:
 - Memory (heap)
 - Global variables
 - Open files
- This makes communication between threads faster than between processes.

➤ **Own Execution Context**

Each thread has its own:

- **Program Counter (PC)** – tracks which instruction is being executed.
- **Stack** – for function calls, local variables, and execution history.
- **Registers** – store temporary data for computations.

➤ **Lightweight**

- Threads are called “lightweight processes” because they use fewer resources than full processes.
- Creating or switching threads is faster than creating or switching processes.

➤ **Concurrent Execution**

- Multiple threads of the same process can run **simultaneously** (on multiple CPU cores) or appear to run simultaneously (time-sharing on a single core).

➤ **Dependent on Process**

- If the parent process is terminated, all its threads are also terminated.

➤ **Communication is Easier**

- Threads can communicate directly through shared memory without complex inter-process communication (IPC) mechanisms.

6.3 Thread States

In an **Operating System**, a **thread can be in one of several states** during its lifecycle, depending on what it is doing. These states are very similar to process states but apply specifically to threads.

New (Created)

- The thread has been **created**, but it **has not started execution yet**.
- Example: A thread object is instantiated in Java but `start()` has not been called.

Runnable (Ready)

- The thread is **ready to run**, waiting for CPU time.
- It may be **waiting in the queue** of the thread scheduler.
- Example: A thread is ready but the CPU is busy running other threads.

Running

- The thread is **currently executing** on the CPU.
- Only one thread per CPU core can run at a time.

Waiting / Blocked

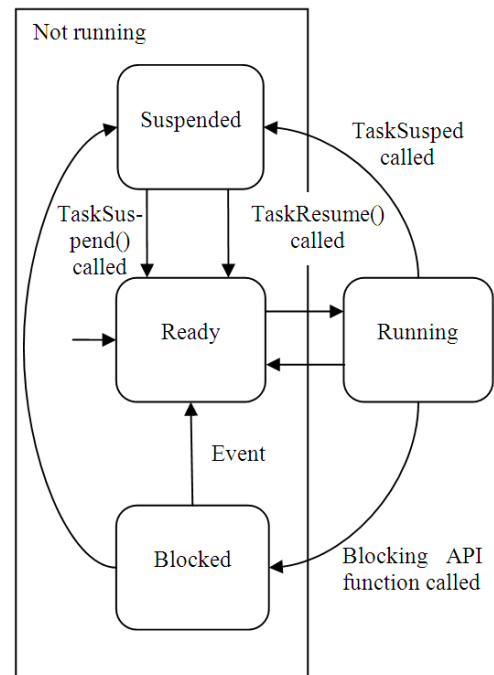
- The thread is **waiting for an event or resource** (like I/O completion or a lock).
- It **cannot run** until the event occurs or the resource becomes available.

Timed Waiting

- Similar to waiting, but the thread will **wait only for a specific time**.
- After the time expires, it either moves to runnable state or continues waiting based on the condition.
- Example: `sleep(milliseconds)` in Java.

Terminated (Dead)

- The thread has **finished execution** or has been **killed**.
- Once terminated, a thread **cannot be restarted**



6.4 Thread Operations

In an **Operating System**, **thread operations** are the actions that can be performed on threads to create, control, or synchronize them. These operations allow threads to execute, coordinate, and terminate properly. Here's a structured explanation:

1. Thread Creation

- A new thread is created within a process.
- Steps usually involve:
 1. Define the task/code for the thread.
 2. Instantiate the thread.
 3. Start the thread so it moves to **runnable state**.
- **Example:** In c language:

```
int main() {  
    pthread_t t; // Thread identifier  
    // Create a new thread, which starts running printMessage  
    pthread_create(&t, NULL, printMessage, NULL); // Wait for the thread to finish  
    pthread_join(t, NULL);  
    return 0;  
}
```

2. Thread Termination

- Ends a thread's execution.
- Happens in three ways:
 1. **Normal completion** – thread finishes its task.
 2. **Explicit stop** – some programming languages provide a stop method (though not recommended).
 3. **Process termination** – if the parent process ends, all its threads terminate.

3. Thread Scheduling

- Determines **which thread will run** on the CPU.
- Threads in the **runnable state** are scheduled according to:
 - **Priority** (high-priority threads may run first)
 - **Time-sharing** (all threads get CPU time in turns)
- OS decides via **thread scheduler**.

4. Thread Synchronization

- Ensures **threads don't interfere** with each other when accessing shared resources.
- Key operations:
 - **Locking / Mutex** – only one thread can access a resource at a time.
 - **Semaphore** – controls access to a resource by multiple threads.
 - **Wait / Notify** – threads can wait for events and get notified when the event occurs.

5. Thread Communication

- Threads within the same process can **communicate directly** because they share memory.
- Methods include:
 - Shared variables (with proper synchronization)

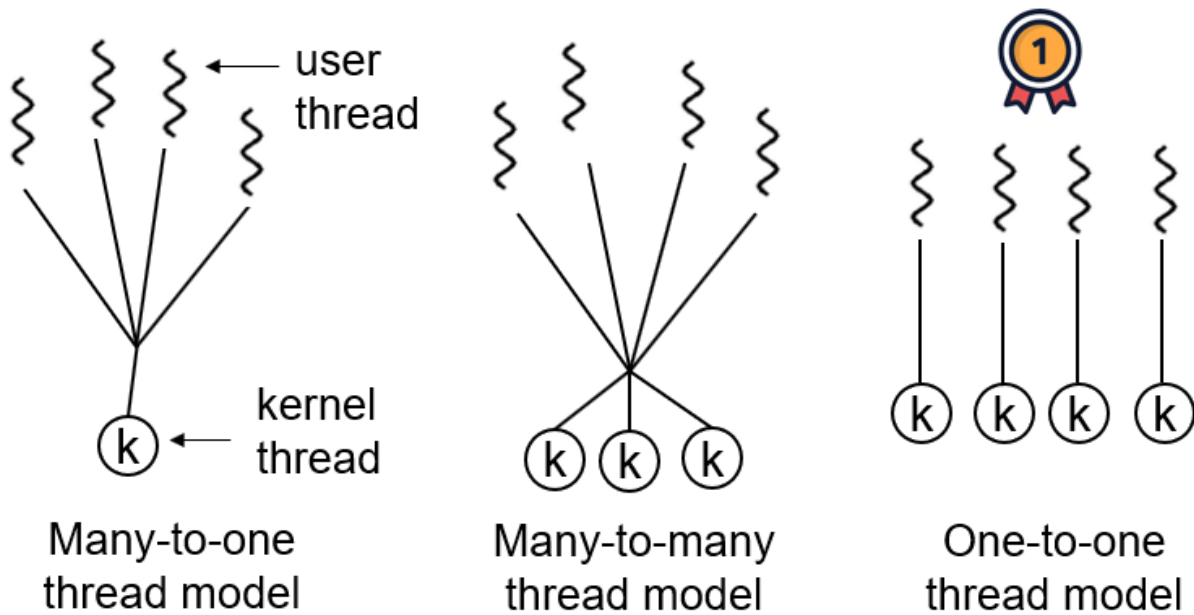
6. Thread Suspension and Resumption

- Some OS or programming languages allow a thread to be **paused (suspended)** temporarily and **resumed** later.
- Example use: Pausing a background task temporarily.

7. Thread Yield

- A thread can **voluntarily give up CPU** to allow other threads to execute.
- Helps improve concurrency and fairness.

Chapter 7: Types of Threads and Threading Models



7.1 UserLevel Threads (ULTs)

User-Level Threads (ULTs) are threads that are **managed entirely by a user-level thread library**, without any kernel support. The **OS kernel is unaware** of these threads; it only sees the process as a single schedulable unit.

- All thread management—creation, scheduling, synchronization—is done in **user space**.
- The kernel only schedules the **process**, not individual threads.

Advantages of ULTs:

- Very fast creation and context switching.
- No kernel involvement → efficient CPU usage.
- Can be implemented on operating systems that **don't support threads**.

Disadvantages of ULTs:

- **Blocking problem:** A single thread blocking blocks the entire process.
- **No true parallelism on multiprocessors:** Since the kernel sees only one process, multiple threads cannot run simultaneously on multiple CPUs.
- **Scheduling** is limited by the library; the kernel cannot intervene.

7.2 KernelLevel Threads (KLTs)

Kernel-Level Threads (KLTs) are threads that are **managed directly by the operating system kernel**.

- The **kernel is aware of every thread** and can schedule them independently.
- Each thread is treated as a separate **schedulable entity** by the OS

Advantages of KLTs:

- **True parallelism** on multiprocessor systems.
- **No blocking problem:** a blocked thread doesn't block other threads in the same process.
- **Better integration** with OS services and scheduling.

Disadvantages of KLTs:

- **Slower thread operations** (creation, context switching) due to kernel involvement.
- **More resource-intensive:** requires kernel memory for thread management.
- **Complexity:** depends on kernel implementation.

7.3 Hybrid (ManytoMany) Model

The **Hybrid Threading Model (Many-to-Many)** is a threading model where:

- **Many user-level threads (ULTs)** are mapped to **many kernel-level threads (KLTs)**.
- The system can create **any number of user threads** and map them to a **smaller or equal number of kernel threads**.
- Combines the advantages of **ULTs** and **KLTs**.

How it Works:

1. A process can have multiple **user-level threads**.
2. The OS kernel manages a pool of **kernel threads**.
3. User threads are **mapped to available kernel threads** for execution.
4. The thread library handles **scheduling user threads** onto kernel threads.

Chapter 8: Advantages of Threads over Processes

1. Faster Creation and Termination

- Creating a thread requires **less overhead** than creating a full process because threads **share the same process resources**.
- Terminating a thread is also faster, as the OS doesn't need to release all process resources.
- Example: In a web browser, opening a new tab as a thread is quicker than starting a whole new process.

2. Lower Context Switching Overhead

- Switching between threads of the same process is **faster than switching between processes**.
- Threads share memory and resources, so the CPU doesn't need to reload the entire process context.
- Example: Switching between multiple threads handling different parts of a program is more efficient.

3. Efficient Communication

- Threads can **communicate directly** through shared memory because they exist in the same process.
- Processes, in contrast, need **inter-process communication (IPC)** mechanisms, which are slower.
- Example: Threads updating a shared buffer do not need pipes or sockets for communication.

4. Better Responsiveness and Concurrency

- Threads allow **parallel execution** within a single process.
- Programs remain **responsive** even if one thread is performing a long task.
- Example: In a media player, one thread plays music while another handles user input.

5. Resource Sharing and Scalability

- Threads **share code, data, and files** of the parent process, which reduces memory usage.
- Creating many threads is **scalable**, allowing programs to perform multiple tasks concurrently without duplicating resources.
- Example: A server handling thousands of client requests with threads instead of creating a new process for each client.

Chapter 9: Challenges in Thread Programming

1. Race Conditions

A **race condition** occurs when two or more threads access shared data concurrently, and the final result depends on the timing or order of execution.

Causes

- Unsynchronized access to shared variables
- Non-atomic operations (read–modify–write)

Example

Two threads incrementing a shared counter without locks may produce incorrect results.

Solutions

- Mutexes / Locks
- Semaphores
- Atomic operations
- Critical sections

2. Deadlocks

A **deadlock** happens when two or more threads wait indefinitely for resources held by each other.

Necessary Conditions (Coffman Conditions)

1. Mutual exclusion
2. Hold and wait
3. No preemption
4. Circular wait

Example

Thread A holds Lock 1 and waits for Lock 2, while Thread B holds Lock 2 and waits for Lock 1.

Prevention Techniques

- Lock ordering
- Avoid hold-and-wait
- Use timeouts
- Deadlock detection and recovery

3. Starvation and Priority Inversion

Starvation

Occurs when a thread never gets CPU or resources because others dominate execution.

Causes

- Poor scheduling policies
- Excessive priority differences

Solution

- Fair scheduling
- Aging (increasing priority over time)

Priority Inversion

Occurs when a **low-priority thread holds a resource needed by a high-priority thread**, while medium-priority threads keep preempting the low-priority thread.

Solution

- Priority inheritance protocol
- Priority ceiling protocol

4. Thread Safety and Synchronization

Thread Safety

A program is **thread-safe** if it behaves correctly when accessed by multiple threads simultaneously.

Synchronization Mechanisms

- Mutexes
- Semaphores
- Read–Write locks
- Condition variables
- Monitors

Challenges

- Performance overhead
- Risk of deadlocks
- Complexity in design

5. Debugging and Testing Multithreaded Programs

Multithreaded bugs are **hard to reproduce** and often non-deterministic.

Common Issues

- Race conditions
- Deadlocks
- Heisenbugs (bugs that disappear during debugging)

Debugging Techniques

- Logging with timestamps and thread IDs
- Thread analyzers and debuggers
- Static analysis tools
- Stress and load testing

6. Thread Cancellation and Signal Handling

Thread Cancellation

Stopping a thread before it completes execution.

Types

- Deferred cancellation (safe points)
- Asynchronous cancellation (dangerous)

Challenges

- Resource leaks
- Inconsistent shared state

Solution

- Cleanup handlers
- Cancellation points

Chapter 10: RealWorld Examples and Case Studies

1. Web Browsers

Use of Threads

Modern web browsers (e.g., Chrome, Firefox) are highly multithreaded:

- **UI thread** – handles user interaction
- **Rendering threads** – layout and paint web pages
- **JavaScript engine threads**
- **Network threads** – handle downloads and HTTP requests
- **GPU threads** – hardware acceleration

Real-World Challenges

- Race conditions between rendering and JavaScript execution
- Deadlocks between UI and background threads
- Security risks from shared memory

Case Study: Google Chrome

- Uses a **multi-process + multi-threaded architecture**
- Each tab runs in a separate process
- Threads inside processes improve responsiveness
- Isolation improves stability and security

2. Database Servers

Use of Threads

Database servers (MySQL, PostgreSQL, Oracle) use threads to:

- Handle multiple client connections
- Execute concurrent queries
- Manage background tasks (logging, backups, replication)

Real-World Challenges

- Data consistency with concurrent transactions
- Deadlocks during table or row locking
- Starvation of long-running queries

Case Study: PostgreSQL

- Uses **MVCC (Multi-Version Concurrency Control)**
- Readers do not block writers
- Background threads manage vacuuming and checkpoints
- Deadlock detection algorithms resolve lock conflicts

3. Operating System Kernels

Use of Threads

OS kernels use threads to:

- Schedule processes
- Handle interrupts
- Manage I/O operations
- Run kernel services

Real-World Challenges

- Race conditions in kernel data structures
- Priority inversion
- Deadlocks in device drivers

Case Study: Linux Kernel

- Uses **spinlocks** for short critical sections
- **Mutexes** for longer operations
- Implements **priority inheritance** to prevent priority inversion
- Supports per-CPU data structures to reduce contention

4. Multimedia and Game Engines

Use of Threads

Game engines (Unity, Unreal Engine) and multimedia apps use threads for:

- Rendering graphics
- Physics simulation
- AI computation
- Audio processing
- Asset loading

Real-World Challenges

- Synchronizing physics and rendering threads
- Maintaining real-time performance (low latency)
- Avoiding frame drops due to locks

Case Study: Unreal Engine

- Dedicated threads for:
 - Game logic
 - Rendering
 - Physics
- Uses **task-based parallelism**
- Lock-free data structures to improve performance

5. Mobile Applications

Use of Threads

Mobile apps (Android, iOS) use threads to:

- Keep the UI responsive
- Perform background tasks (network, database, sensors)
- Handle push notifications

Real-World Challenges

- UI thread blocking (causing app freezes)
- Battery and CPU constraints
- Race conditions in shared resources

Case Study: Android Applications

- Main/UI thread handles user interaction
- Background work handled using:
 - Thread pools
 - AsyncTask (deprecated)
 - Executors
 - Kotlin Coroutines
- Strict UI thread rules prevent unsafe updates

Chapter 11: Process vs Thread:

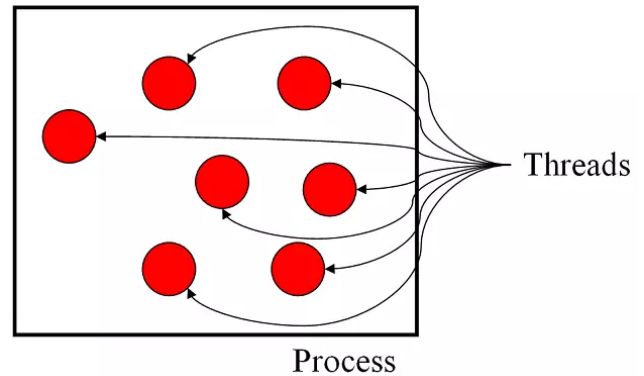
Aspect	Process	Thread
Definition	A process is an independent program in execution	A thread is a lightweight unit of execution within a process
Address Space	Has its own separate address space	Shares the same address space with other threads
Creation Time	Slower (requires more system resources)	Faster compared to process creation
Resource Ownership	Owns resources like memory, files, and I/O	Shares resources of the process
Communication	Inter-process communication (IPC) needed (pipes, sockets, shared memory)	Direct communication through shared memory
Context Switching	Heavyweight and time-consuming	Lightweight and faster
Isolation	High isolation; failure doesn't affect others	Low isolation; failure may affect entire process
Scheduling	Scheduled independently by OS	Scheduled as part of the parent process
Overhead	High overhead	Low overhead
Security	More secure due to isolation	Less secure due to shared resources
Blocking	One process blocking does not affect others	One thread blocking may block the process
Example	Running a text editor and browser	Multiple threads in a web browser tab

Chapter 12: Conclusion and Future Trends in Process and Threads

Conclusion

Processes and threads are fundamental concepts in modern operating systems and software development.

Processes provide isolation, security, and stability by executing programs in separate memory spaces, while **threads** enable efficient concurrency and parallelism by sharing resources within a process. Together, they allow systems to achieve responsiveness, scalability, and optimal utilization of hardware resources.



However, using processes and threads introduces challenges such as synchronization overhead, race conditions, deadlocks, and debugging complexity. Effective design, proper synchronization mechanisms, and careful scheduling are essential to harness their benefits while minimizing risks.

Future Trends in Processes and Threads

1. Increasing Core Counts and Parallelism

As multi-core and many-core processors become standard, software will rely more heavily on multithreading and parallel execution to achieve performance gains.

2. Lightweight Concurrency Models

Emerging models such as **green threads**, **fibers**, and **coroutines** reduce overhead compared to traditional OS threads and simplify concurrent programming.

3. Task-Based and Asynchronous Programming

Future systems emphasize **task-based parallelism** and **async/await** models, allowing developers to focus on tasks rather than low-level thread management.

4. Improved Scheduling and Resource Management

Operating systems will adopt smarter schedulers using **AI-driven optimization** and **energy-aware scheduling**, especially for mobile and embedded devices.

5. Stronger Isolation with Efficient Communication

Hybrid models combining process isolation with fast inter-process communication (e.g., containers and microservices) will become more common.

6. Better Tooling and Debugging Support

Advanced debuggers, race detectors, and formal verification tools will improve the reliability of multithreaded applications.

Final References (IEEE Standard – Clean Version)

Books

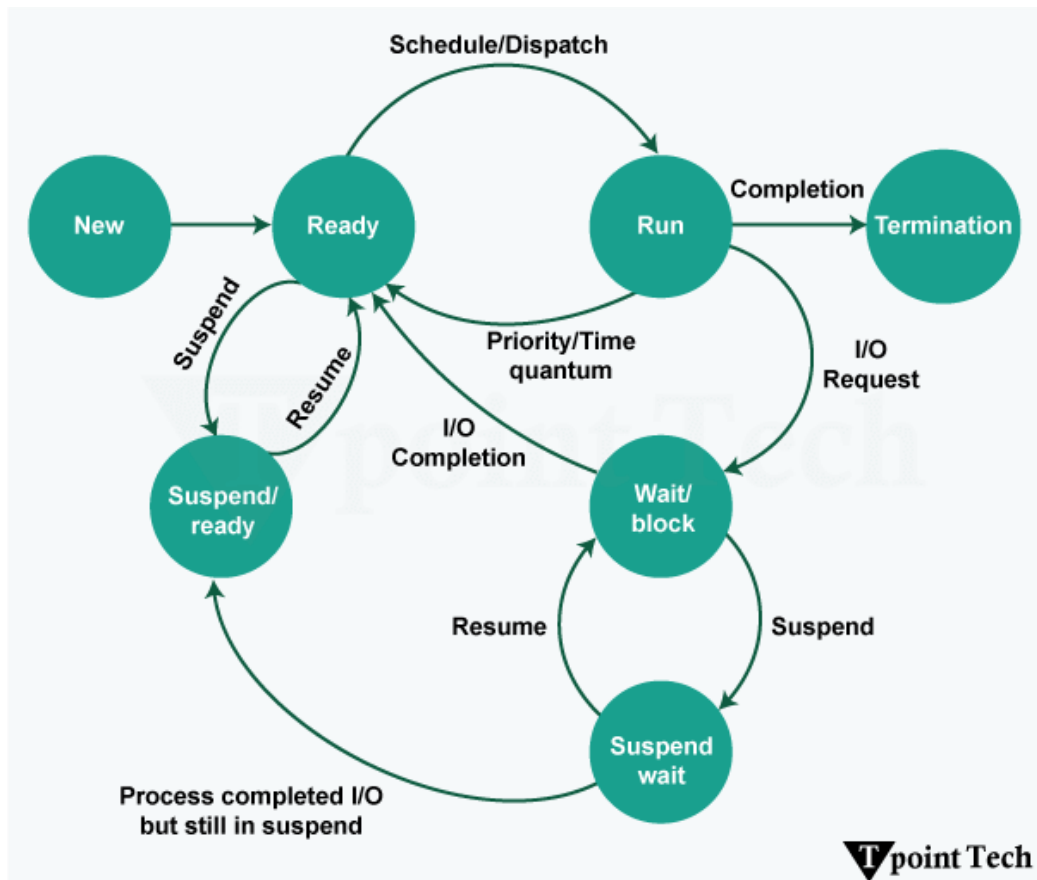
- [1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: John Wiley & Sons, 2020.
- [2] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Boston, MA, USA: Pearson Education, 2015.

Trusted Online Educational Sources

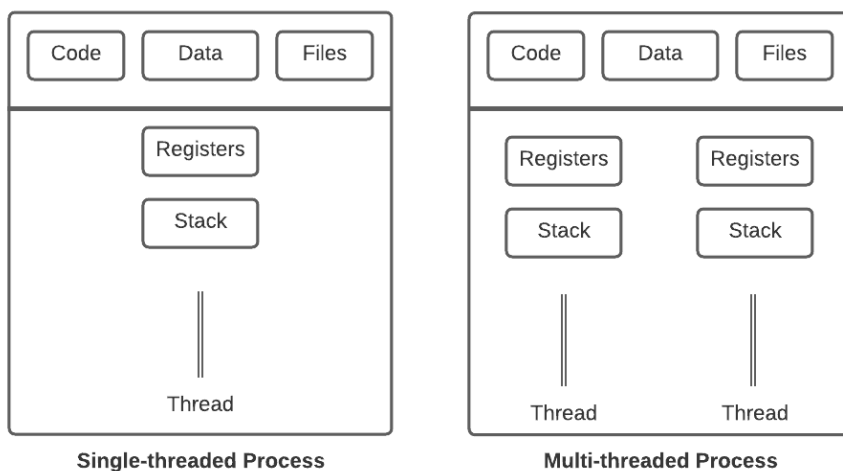
- [3] GeeksforGeeks, *Process Management in Operating Systems*.
<https://www.geeksforgeeks.org/process-management/>
- [4] GeeksforGeeks, *Process Control Block (PCB) in Operating Systems*.
<https://www.geeksforgeeks.org/process-control-block/>
- [5] GeeksforGeeks, *Inter-Process Communication (IPC) in Operating Systems*.
<https://www.geeksforgeeks.org/inter-process-communication/>
- [6] GeeksforGeeks, *Threads in Operating Systems*.
<https://www.geeksforgeeks.org/thread-in-operating-system/>
- [7] TutorialsPoint, *Operating System – Processes*.
https://www.tutorialspoint.com/operating_system/os_processes.htm
- [8] TutorialsPoint, *Operating System – Threads*.
https://www.tutorialspoint.com/operating_system/os_threads.htm
- [9] TutorialsPoint, *Operating System – Inter-Process Communication*.
https://www.tutorialspoint.com/operating_system/os_interprocess_communication.htm
- [10] StudyTonight, *Operating System – Process Management*.
<https://www.studytonight.com/operating-system/process-management>
- [11] StudyTonight, *Operating System – Threads*.
<https://www.studytonight.com/operating-system/threads>
- [12] TutorialsTeacher, *Operating System – Process States*.
<https://www.tutorialsteacher.com/operating-system/process-states>
- [13] Javatpoint, *Operating System – Process vs Thread*.
<https://www.javatpoint.com/process-vs-thread>

Appendix : Diagrams

1. Process state diagram.

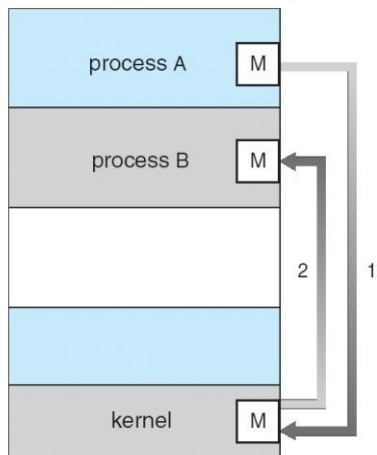


2. Single threaded vs multi threaded process



3. IPC mechanisms diagram.

Message Passing



Shared Memory

